servidor.py

Servidor local: entrega o site, responde à API e liga a IA com a chave do .env.
Python · mais de 440 linhas · guia dos blocos principais

Visão geral

O **servidor.py** abre um servidor em *http://localhost:8000*. Ele faz duas coisas: **serve os arquivos do site** (index.html, script.js, dados...) e atende a **API** que o site chama. Isso permite usar o Gemini com segurança (a chave fica no servidor, num arquivo *.env* fora do GitHub) e atualizar as correntes pelo botão do mapa.

Endereço	O que faz
`/api/route`	Calcula as 3 rotas e o parecer do Gemini (usa *ai_agent.compute_three_routes*).
`/api/chat`	Conversa com o Copiloto (ferramentas de vazamento e cardumes).
`/api/fish-hotspots`	Previsão global de cardumes.
`/api/status`	Estado do download de correntes.
`/api/atualizar`	Roda o fetch_ocean.py para baixar correntes novas (botão do mapa).

Este guia cobre os blocos principais que pude ler do arquivo; o tratamento de requisições e do botão "Atualizar correntes" não está detalhado aqui.

Blocos de código explicados

Bloco 1 — Leitura do arquivo .env (a chave)

servidor.py · Python

```
def _load_env_file():
    """Carrega ROOT/.env (GEMINI_API_KEY, OPENAI_API_KEY...) sem sobrescrever variaveis ja
    definidas no ambiente. Usa python-dotenv se instalado; senao, um leitor simples KEY=VALOR."""
    env_path = ROOT / ".env"
    if not env_path.is_file():
        return
    try:
        from dotenv import load_dotenv
        load_dotenv(env_path, override=False)
        return
    except ImportError:
        pass
    try:
        for line in env_path.read_text(encoding="utf-8-sig").splitlines():
            line = line.strip()
            if not line or line.startswith("#") or "=" not in line:
                continue
            key, value = line.split("=", 1)
            key = key.strip()
            if key.startswith("export "):
                key = key[len("export "):].strip()
            value = value.strip().strip('"').strip("'")
            if key and key not in os.environ:
                os.environ[key] = value
    except OSError as e:
        print(f"  aviso: nao consegui ler {env_path}: {e}")


_load_env_file()

import ai_agent  # noqa: E402  (depois do .env: o agente le as chaves do ambiente)
```

O que este bloco faz

Procura o *.env* ao lado do servidor.py e coloca cada linha *CHAVE=valor* nas variáveis de ambiente. Se o *python-dotenv* estiver instalado, usa ele; senão, usa um leitor simples que ignora linhas vazias e comentários (#) e remove aspas.

Variáveis que já existem no sistema **não são sobrescritas**. O **import ai_agent** vem depois de propósito: o agente lê as chaves do ambiente, então o .env precisa ser carregado antes.

Bloco 2 — Parecer do Gemini para a rota

servidor.py · Python

```
def gerar_analise_gemini(dados_rota):
    api_key = os.getenv('GEMINI_API_KEY')
    if not api_key:
        return None

    # Identificadores ativos, em ordem de tentativa
    modelos_suportados = [
        'gemini-flash-latest',
        'gemini-3.8-flash',
        'gemini-flash-lite-latest',
    ]

    prompt = f"""
    Atue como um Especialista em Logística Marítima.
    Análise da rota {dados_rota.get('origem')} -> {dados_rota.get('destino')}:
    - Rota Comercial: {dados_rota.get('dist_comercial')} NM, {dados_rota.get('combustivel_comercial')} t VLSFO.
    - Copiloto IA (100% Eco): {dados_rota.get('dist_ia')} NM, {dados_rota.get('combustivel_ia')} t VLSFO (...).

    Forneça um parecer técnico sucinto (máximo 3 frases) destacando os benefícios de combustível e emissões.
    """

    if genai is None:
        # SDK ausente: usa a chamada REST direta do ai_agent (nao precisa de pip install)
        texto = ai_agent.call_llm_text(prompt)
        if texto and texto.strip():
            print("[GEMINI SUCESSO] Resposta gerada via REST")
            return texto.strip()
        return None

    genai.configure(api_key=api_key)
    ...  # tenta cada modelo pela SDK; se todos falharem, tenta a chamada REST
```

O que este bloco faz

Monta o pedido ao Gemini com os números da rota (distâncias e combustível) e pede um parecer de até 3 frases. Sem chave, devolve *None* (a rota usa o texto local).

Plano B importante: se a biblioteca *google-generativeai* não estiver instalada, o servidor usa a chamada REST do *ai_agent.call_llm_text*, que não precisa de nenhum *pip install*. Foi isso que resolveu o erro “LLM indisponível”.

Bloco 3 — Liga o parecer ao cálculo das rotas

servidor.py · Python

```
def _parecer_rota(origem, destino):
    """Adapta os números de ai_agent.compute_three_routes para o dicionário de gerar_analise_gemini."""
    def gerar(comm_m, astar_m, ai_m, scenario):
        economia_t = max(0.0, comm_m["fuel_tons"] - ai_m["fuel_tons"])
        dados_rota = {
            "origem": origem,
            "destino": destino,
            "dist_comercial": round(comm_m["km"] * KM_TO_NM),
            "combustivel_comercial": round(comm_m["fuel_tons"], 1),
            "dist_ia": round(ai_m["km"] * KM_TO_NM),
            "combustivel_ia": round(ai_m["fuel_tons"], 1),
            "economia_usd": round(economia_t * ai_agent.FUEL_PRICE_USD),
            "co2_evitado": round(economia_t * ai_agent.CO2_PER_TON_FUEL, 1),
        }
        return gerar_analise_gemini(dados_rota)
    return gerar
```

O que este bloco faz

Função que devolve outra função. O *ai_agent* calcula as rotas e, ao final, chama esta função com as métricas. Ela converte km em milhas náuticas, calcula a economia em toneladas, dólares e CO_2 e entrega tudo ao Gemini.

Bloco 4 — Mensagens de partida e abrir o navegador

servidor.py · Python

```
    gemini_key = os.getenv("GEMINI_API_KEY", "")
    openai_key = os.environ.get("OPENAI_API_KEY")
    if gemini_key:
        print(f"  Copiloto IA: chave GEMINI_API_KEY detectada (...{gemini_key[-4:]}) — usando Gemini.")
    elif openai_key:
        print(f"  Copiloto IA: chave OPENAI_API_KEY detectada (...{openai_key[-4:]}) — usando OpenAI.")
    else:
        print("  Copiloto IA: nenhuma chave (GEMINI_API_KEY/OPENAI_API_KEY) no ambiente nem no .env —")
        print("  usando simulação otimizada offline (rotas e chat funcionam normalmente,")
        print("  com o ganho de eficiência da Rota IA calculado deterministicamente).")
    print("=" * 52)
    if not os.environ.get("SEM_NAVEGADOR"):  # inicio automatico com o Windows: nao abre o navegador
        try:
            webbrowser.open(url)
        except Exception:
            pass
```

O que este bloco faz

Ao iniciar, o terminal informa qual IA está ativa. Por segurança, **só mostra os 4 últimos caracteres da chave** (*...2bBw*), o suficiente para conferir qual chave foi lida. Sem chave, avisa que ficará no modo offline.

Depois abre o navegador — exceto se a variável **SEM_NAVEGADOR** estiver definida (usada pelo início automático do Windows).

Bloco 5 — Registro do que o servidor mostra (log)

servidor.py · Python

```
class _Tee:
    """Duplica o que sai no terminal em servidor_log.txt (diagnostico do Gemini)."""
    def __init__(self, stream, path):
        self._s = stream
        self._f = open(path, "a", encoding="utf-8", buffering=1)
    def write(self, data):
        try:
            self._s.write(data)
        except Exception:
            pass
        self._f.write(data)
        return len(data)
    def flush(self):
        try:
            self._s.flush()
        except Exception:
            pass
        self._f.flush()
    def __getattr__(self, name):
        return getattr(self._s, name)


try:
    sys.stdout = _Tee(sys.stdout, ROOT / "servidor_log.txt")
    sys.stderr = _Tee(sys.stderr, ROOT / "servidor_log.txt")
    print("\n===== servidor iniciado", time.strftime("%Y-%m-%d %H:%M:%S"), "=====")
except Exception:
    pass
```

O que este bloco faz

Foi acrescentado para diagnosticar o Gemini: tudo o que o servidor imprime passa também para o arquivo *servidor_log.txt*. A classe **_Tee** “duplica” a saída: escreve no terminal e no arquivo. Com o servidor rodando escondido (sem janela), é o único lugar onde os avisos e erros aparecem. O arquivo está no *.gitignore*.